

The Foundations under “Kasami is Almost Bent”

A Picture Book of the Basic Building Blocks

Abstract

This note shows the *lowest* pieces that hold up a Lean 4 proof: the Kasami power map $x \mapsto x^d$ is *Almost Bent*.

The proof stands on **two pillars** that meet at the top. Each pillar stands on its own **leaf** that touches Mathlib directly. We draw the shape first, then walk up it.

Every block gets a small picture and the matching Lean snippet. The English is kept plain on purpose.

1 The shape of the proof

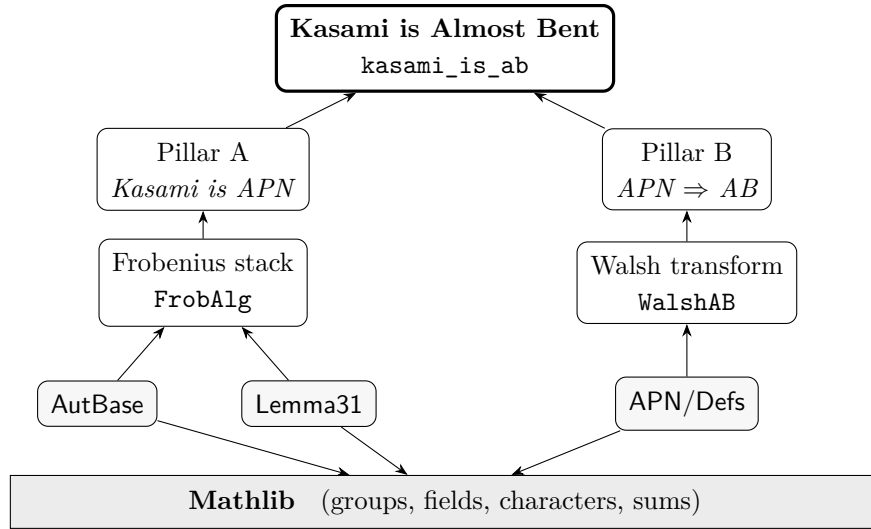
The whole proof is a small tree.

At the bottom are three **leaves**. A leaf is a file that imports *only* Mathlib. So a leaf sits straight on top of Mathlib, with nothing in between.

The three leaves are

AutBase Lemma31 APN/Defs.

Above the leaves are **two pillars**.



Read the picture bottom to top.

AutBase feeds only the left pillar (Kasami is APN), through the Frobenius stack.

APN/Defs feeds the right pillar ($APN \Rightarrow AB$), through the Walsh transform. This path *skips* AutBase.

So the top theorem reaches Mathlib in two ways at once: partly through AutBase, partly on its own. That is the one fact to keep in mind.

2 Leaf 1 — AutBase: a twisted multiply

The first leaf studies one very simple map. Pick a constant a . Raise the input to a power of the characteristic, then multiply by a :

$$T_r(a): \quad x \mapsto a \cdot x^{p^r}.$$

Because $x \mapsto x^p$ is the Frobenius map, $T_r(a)$ is “Frobenius, then scale”. These maps are the alphabet of the whole APN pillar.

$$\begin{array}{ccccc} x & \xrightarrow{x^{p^r}} & x^{p^r} & \xrightarrow{\cdot a} & a x^{p^r} \\ & \searrow & & \nearrow & \\ & & T_r(a) & & \end{array}$$

```
-- RequestProject/FiniteField/AutBase.lean (imports only Mathlib)
def semilinearOp (r : ℕ) (a : F) (x : F) : F := a * x ^ (p ^ r)
```

We also track *which* powers actually show up in an additive polynomial. That set of indices is the *support*.

```
noncomputable def support (n : ℕ) (coeffs : Fin n → F) : Finset (Fin n) :=
  Finset.univ.filter fun i => coeffs i ≠ 0
```

3 Leaf 2 — Lemma31: turn a product into a difference

The second leaf is one clean trick.

Take a linear map L and a multiplicative map M . Look at the product map

$$P(x) = L(x) \cdot M(x).$$

The trick: P is injective exactly when a family of difference maps are all bijective. This swaps a hard product question for an easy linear one.

$P = L \cdot M$ injective

$\iff$

$\Delta_{y_1} - \Delta_{y_2}$ bijective
for all $y_1 \neq y_2$

```
-- RequestProject/FiniteField/Lemma31.lean (imports only Mathlib)
def PMap31 (L : F →1[K] F) (M : F → F) (x : F) : F := L x * M x

lemma PMap31_inj_iff_Delta_sub_bij (L : F →1[K] F) {M : F → F}
  (hM : ∀ a b, M (a * b) = M a * M b) (hMinj : Function.Injective M) :
  Function.Injective (PMap31 L M) ↔
  ∀ y₁ y₂ : F, y₁ ≠ y₂ → Function.Bijective (Delta L M y₁ - Delta L M y₂)
```

4 Leaf 3 — APN/Defs: the Kasami exponent

The third leaf just *names* the objects of the right pillar.

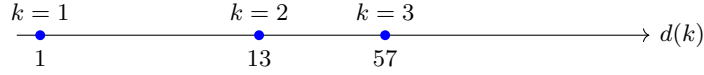
The star is the **Kasami exponent**

$$d(k) = 2^{2k} - 2^k + 1.$$

The helper is the **linearized polynomial**

$$L_k(x) = x^{2^k} + x,$$

which is additive: $L_k(x + y) = L_k(x) + L_k(y)$.



The first Kasami exponents: $d(1) = 1$, $d(2) = 13$, $d(3) = 57$.

```
-- RequestProject/APN/Defs.lean (imports only Mathlib)
def d (k : ℕ) : ℕ := 2 ^ (2 * k) - 2 ^ k + 1

def L (k : ℕ) (x : F) : F := x ^ (2 ^ k) + x
```

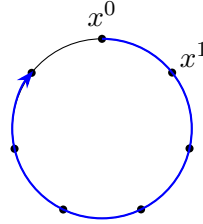
5 The glue — Frobenius arithmetic

Both pillars lean on one fact about finite fields.

A nonzero element x satisfies $x^{|F|-1} = 1$. So an exponent only matters *modulo* $|F| - 1$:

$$a \equiv b \pmod{|F| - 1} \implies x^a = x^b.$$

This is the wrap-around that lets us slide exponents freely.



exponents live on a circle of length $|F| - 1$

```
-- RequestProject/FiniteField/FrobAlg.lean (the Dempwolff--Müller stack)
lemma pow_eq_pow_of_mod_eq {x : F} (hx : x ≠ 0) {a b : ℕ}
  (hab : a % (Fintype.card F - 1) = b % (Fintype.card F - 1)) :
  x ^ a = x ^ b
```

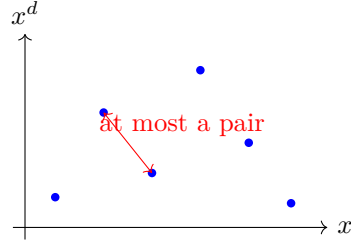
6 Pillar A — Kasami is APN

APN means: in every nonzero direction a , the difference equation

$$(x + a)^d + x^d = b$$

has at most two solutions.

The proof of this pillar uses AutBase (the twisted multiplies) and the Frobenius glue. It shows two distinct inputs cannot collide.



APN: no nonzero direction sends three inputs to the same value.

```
-- RequestProject/Kasami/Classical/KasamiAPN.lean
theorem kasami_is_apn {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {n : ℕ} (hn : Fintype.card F = 2 ^ n) (k : ℕ)
  (hk : 1 < k) (hk_odd : Odd k) (hkn : k < n)
  (hn_odd : Odd n) (hcop : Nat.Coprime k n) :
  IsAPN (fun (x : F) => x ^ (kasamiExp k))
```

7 Pillar B — the Walsh transform and AB

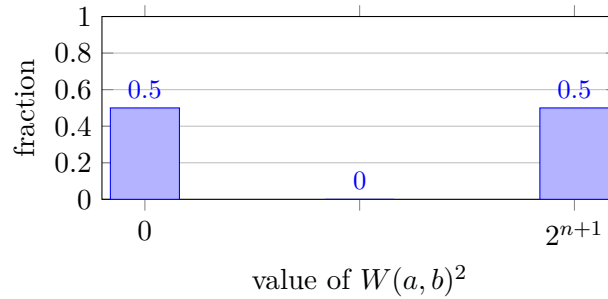
The right pillar measures the map with **characters**.

Let $\chi(x) = +1$ if the trace of x is 0, else -1 . The **Walsh coefficient** sums this sign over all inputs:

$$W(a, b) = \sum_x \chi(ax + bf(x)).$$

A map is **Almost Bent (AB)** when every squared Walsh value is flat — only two bars are allowed:

$$W(a, b)^2 \in \{0, 2^{n+1}\} \quad (a \neq 0).$$



An AB map: the Walsh spectrum has only two heights.

```
-- RequestProject/Walsh/WalshAB.lean (rooted in Mathlib via APN/Defs)
noncomputable def walsh (f : F → F) (a b : F) : ℤ :=
  ∑ x : F, χ (a * x + b * f x)

noncomputable def IsAB {n : ℕ} ( _ : Fintype.card F = 2 ^ n) (f : F → F) : Prop :=
  ∀ a : F, a ≠ 0 → ∀ b : F,
    walsh f a b ^ 2 = 0 ∨ walsh f a b ^ 2 = (2 ^ (n + 1) : ℤ)
```

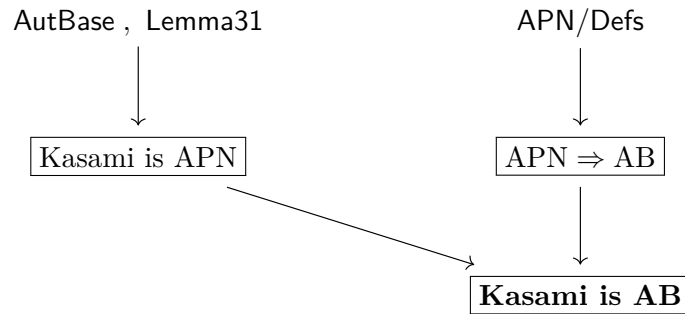
The pillar itself is the step $APN \Rightarrow AB$ for n odd. It works because the fourth moment of the Walsh values is pinned by the APN count, and each squared value is divisible by 2^{n+1} , so it can only be 0 or 2^{n+1} .

```
-- RequestProject/Kasami/Classical/KasamiABProof.lean
theorem apn_power_implies_ab
  {n : ℕ} (hn : Fintype.card F = 2 ^ n) (hn_odd : Odd n) (hn_pos : 1 < n)
  (d : ℕ) (hd : Nat.Coprime d (2 ^ n - 1)) (hd_pos : 0 < d)
  (h_apn : KasamiAPN.IsAPN (fun x : F => x ^ d)) :
  IsAlmostBent hn (fun x : F => x ^ d)
```

8 The top — the two pillars meet

Now join the pillars.

Pillar A gives “Kasami is APN”. Pillar B turns any APN power map into an AB map. Put them together and the crown follows.



```
-- RequestProject/Kasami/Classical/KasamiABProof.lean
theorem kasami_is_ab_proven
  {n : ℕ} (hn : Fintype.card F = 2 ^ n) (k : ℕ)
  (hk : 1 < k) (hkn : k < n)
```

```

(hn_odd : Odd n) (hcop : Nat.Coprime k n) :
  IsAlmostBent hn (fun (x : F) => x ^ (kasamiExp k)) :=
  apn_power_implies_ab hn hn_odd (by linarith) (kasamiExp k)
  (kasami_exp_coprime_general hk hkn hcop hn_odd)
  (by unfold kasamiExp; omega)
  (kasami_is_apn_general hn k hk hkn hn_odd hcop)

```

9 One sentence to remember

AutBase is a real Mathlib-rooted leaf — but it is *not* the single base.

There are three leaves. AutBase (with Lemma31) holds up the APN pillar. APN/Defs holds up the AB pillar, skipping AutBase.

So the top AB theorem rests on Mathlib *partly through AutBase, and partly on its own*.

Three leaves, two pillars, one crown.